

COLECTIVAS PARALELISMO:

- **MPI_Send:** Envía un mensaje al destino indicado.

```
int MPI_Send ( void * buff , int count , MPI_Datatype datatype , int dest , int tag ,  
MPI_Comm comm );
```

- Buff: Buffer de envío.
- Count: Número de elementos del buffer que se envían.
- Datatype: Tipo de datos.
- Dest: Proceso de destino.
- Tag: Etiqueta del mensaje.
- Comm: Comunicador.

- **MPI_Recv:** Recibe un mensaje desde el destino indicado.

```
int MPI_Recv ( void * buff , int count , MPI_Datatype datatype , int source , int tag ,  
MPI_Comm comm , MPI_Status * status );
```

- Buff: Buffer donde se recibe el mensaje.
- Count: Número de elementos que se reciben
- Datatype: Tipo de datos.
- Source: Proceso del que recibe el mensaje
- Tag: Etiqueta del mensaje que recibe.
- Comm: Comunicador
- Status:

- **MPI_Barrier:** Establece una barrera que bloquea el programa hasta que todos los procesos han alcanzado esta rutina.

```
int MPI_Barrier ( MPI_Comm comm );
```

- Comm: Comunicador.

- **MPI_Bcast:** Comunicación uno a todos de desde el proceso root al resto de procesos del comunicador.

```
int MPI_Bcast ( void * buff , int count , MPI_Datatype datatype , int root , MPI_Comm  
comm );
```

- Buff: Buffer que se envía.
- Count: Número de elementos del buffer a enviar.
- Datatype: Tipo de datos.
- Root: Proceso desde el que se envía el buffer.
- Comm: Comunicador.

- **MPI_Scatter:** Distribuye los elementos del proceso root al resto de los procesos del comunicador.

`MPI_Scatter ( void * buff , int sendcnt , MPI_Datatype sendtype , void * recvbuff , int recvcnt , MPI_Datatype recvtype , int root , MPI_Comm comm );`

- Buff: Buffer a distribuir.
- Sendcnt: Número de elementos a distribuir.
- Sendtype: Tipo de datos del buffer.
- Recvbuff: Buffer donde se reciben los elementos en cada proceso.
- Recvcnt: Número de elementos a recibir (coincide con Sendcnt).
- Recvtype: Tipo de datos del buffer de recepción (coincide con Sendtype).
- Root: Proceso que envía el buffer.
- Comm: Comunicador.

- **MPI_Gather:** Recibe en el proceso root elementos desde todos los procesos del comunicador y los almacena.

`MPI_Gather ( void * buff , int sendcnt , MPI_Datatype sendtype , void * recvbuff , int recvcnt , MPI_Datatype recvtype , int root , MPI_Comm comm );`

- Buff: Buffer desde el que se envían los elementos en cada proceso.
- Sendcnt: Número de elementos que se envían.
- Sendtype: Tipo de datos que se envían.
- Recvbuff: Buffer donde se reciben los elementos.
- Recvcnt: Número de elementos que se reciben desde cada proceso. (coincide con Sendcnt).
- Recvtype: Tipo de datos que se reciben (coincide con Sendtype).
- Root: Proceso que recibe los datos.
- Comm: Comunicador.

- **MPI_Reduce:** Realiza una reducción todos a uno, guardando el resultado en el proceso root.

`int MPI_Reduce ( void * buff , void * recvbuff , int count , MPI_Datatype datatype , MPI_Op op , int root , MPI_Comm comm );`

- Buff: Datos de cada proceso a reducir.
- Recvbuff: Buffer donde se reducen los datos.
- Count: Número de elementos a reducir en cada proceso.
- Datatype: Tipo de datos.
- Op: Operación
- Root: Proceso que recibe la reducción de los datos.
- Comm: Comunicador.

- **MPI_Scan:** Cada proceso P_i calcula la reducción de los valores desde el proceso 0 hasta el i , inclusive. Esto significa que el proceso n tendrá la reducción total de todos los procesos.

`int MPI_Scan(void* sendbuff, void* recvbuff, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)`

- Sendbuff: Buffer desde el que se envían los datos en cada proceso.
- Recvbuff: Buffer donde se guarda la reducción.
- Count: Número de elementos a reducir en cada buffer.
- Datatype: Tipo de datos.
- Op: Operación.
- Comm: Comunicador.